

Futures & Options Trading, Volatility and Settlement Management Database System

Database Management System Project

Presented by: Anand Bhausahab Harak **Roll No.:** P06

Class: T.Y. B.Tech CSE (Data Science) **Subject:** Database Management Systems

RDBMS: MySQL **Academic Year:** 2026–27



A relational database to manage F&O contracts, prices, volatility and settlement workflows using NSE datasets. Data cleaned with Python; loaded and analyzed in MySQL.

Problem Statement & Objectives

Problem

NSE F&O datasets contain raw prices, contracts, trading activity and volatility metrics that need structured processing before analysis.

- Heterogeneous raw data needing cleaning & standardization
- Multiple related datasets must map to normalized tables
- Integrity, performance and maintainability are essential

Objectives

- Clean raw F&O data using Python pipelines
- Design normalized MySQL schema (fno_tae2_db)
- Load data into four related tables with constraints
- Implement views, procedures, triggers and optimized queries

Bottom line

Raw Data →

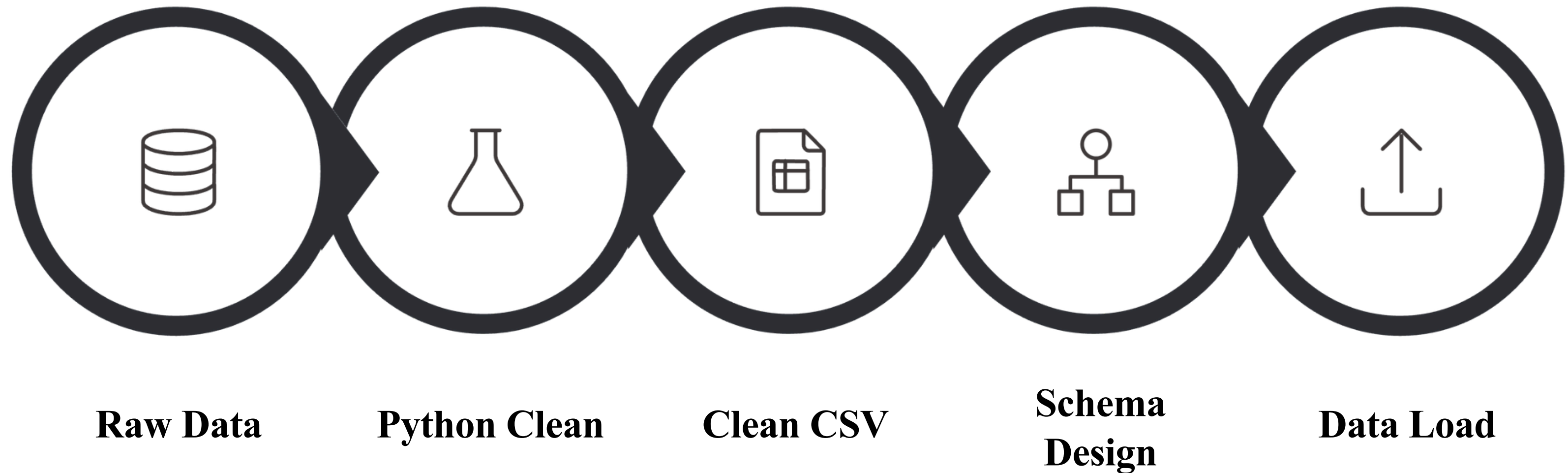
Clean Data →

Relational Database →

SQL Analysis →

Optimization

Project Workflow — From Raw Dataset to Database



This pipeline ensures reproducible ingestion, validated relational modeling and performance-tuned analysis for F&O research and reporting.

Data Cleaning Using Python

The Python cleaning script converts noisy NSE F&O raw files into structured CSVs ready for SQL ingestion. Emphasis: correctness and auditability — no fabricated market values.

- Header & value trimming; standardized column names
- Consistent missing-value tokens (NaN/NULL/N/A/- → NULL)
- Date standardization: EXP_DATE → YYYY-MM-DD
- Numeric validation for prices, OI, trade counts and values
- Drop records missing SYMBOL or EXP_DATE; remove duplicates
- OHLC sanity checks ($\text{Open} \leq \text{High}$, $\text{Low} \leq \text{Close}$ where applicable)

Output artifacts

Cleaned CSVs, validation logs, and populate.py which emits data.sql for MySQL load.

Principle

No imputation of missing market values; keep data provenance traceable.

Database Design & Schema

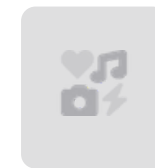
Four core tables with enforced referential integrity

Integrity constraints: PRIMARY KEY, FOREIGN KEY (with CASCADE where appropriate), NOT NULL, UNIQUE and CHECK for volatility ranges.



data1 — Price Data

Underlying & futures price records; primary key data1_id.



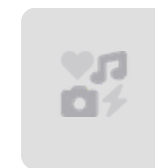
data2 — Volatility Metrics

Calculated volatility values linked to data1_id (one-to-one).



data3 — Contract & Trading

Derivative contract details, trade counts, trade_value; primary key data3_id.



data4 — Derivative OHLC

OHLC pricing per contract linked to data3_id (one-to-one) with audit trail support.

Normalization: 1NF → 2NF → 3NF

01

1NF — Atomicity

Each column holds single (atomic) values: one symbol / trade date / OHLC per row.

02

2NF — Eliminate Partial Dependencies

Non-key attributes stored with the table whose primary key they depend on (data2 linked to data1_id; data4 linked to data3_id).

03

3NF — Remove Transitive Dependencies

No attribute depends on non-key attributes; facts separated into data1-data4 to reduce redundancy and improve integrity.

Result: compact schema, easier maintenance, and reliable joins for analytics.

Data Population & SQL Operations

After cleaning, populate.py generates data.sql which inserts records into the MySQL schema.

Table	Records
data1	220
data2	220
data3	638
data4	638

- **SQL Features Implemented**

- **DDL:**

- CREATE DATABASE/TABLE + constraints

- **DML:**

- INSERT / UPDATE / DELETE

- **DQL:**

- SELECT with ORDER BY / LIMIT

- **DCL / TCL:**

- GRANT, TRANSACTIONS, SAVEPOINTS, ROLLBACK

Advanced SQL: JOINS & Correlated Subqueries

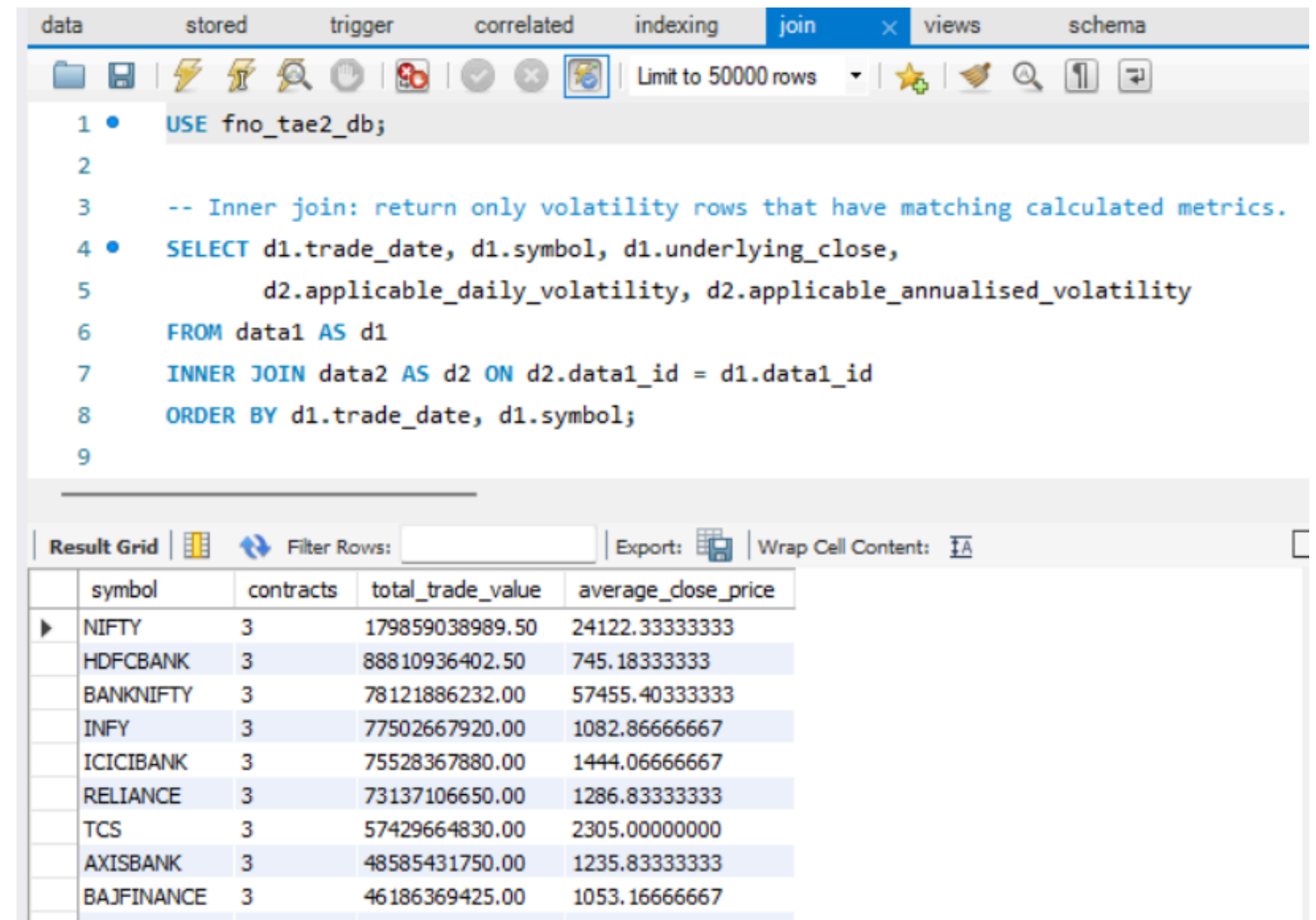
JOIN Patterns

- **INNER JOIN** data1 ↔ data2 (price + volatility)
- **INNER JOIN** data3 ↔ data4 (contract + OHLC)
- **LEFT / RIGHT / UNION** used to emulate FULL OUTER JOIN
- **SELF JOIN** data3 vs data3 to compare expiries

Correlated Subqueries

Per-record comparisons (e.g., contract trade_value vs average trade_value for same symbol; annualised volatility vs trade-date average).

Use JOINS to combine related entities; use correlated subqueries to compare individual rows against group-level metrics for richer analytics.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor has tabs for 'data', 'stored', 'trigger', 'correlated', 'indexing', 'join', 'views', and 'schema'. The 'join' tab is active. The query text is as follows:

```
1 • USE fno_tae2_db;
2
3 -- Inner join: return only volatility rows that have matching calculated metrics.
4 • SELECT d1.trade_date, d1.symbol, d1.underlying_close,
5         d2.applicable_daily_volatility, d2.applicable_annualised_volatility
6 FROM data1 AS d1
7 INNER JOIN data2 AS d2 ON d2.data1_id = d1.data1_id
8 ORDER BY d1.trade_date, d1.symbol;
9
```

The result grid shows the following data:

	symbol	contracts	total_trade_value	average_close_price
▶	NIFTY	3	179859038989.50	24122.33333333
	HDFCBANK	3	88810936402.50	745.18333333
	BANKNIFTY	3	78121886232.00	57455.40333333
	INFY	3	77502667920.00	1082.86666667
	ICICIBANK	3	75528367880.00	1444.06666667
	RELIANCE	3	73137106650.00	1286.83333333
	TCS	3	57429664830.00	2305.00000000
	AXISBANK	3	48585431750.00	1235.83333333
	BAJFINANCE	3	46186369425.00	1053.16666667

Views, Stored Procedure & Trigger

Views

vw_volatility_report (data1 + data2): reusable volatility dataset.

vw_derivative_summary (data3 + data4): grouped summaries with COUNT, SUM, AVG.

Stored Procedure

sp_symbol_report(p_symbol): returns contract count, first/last expiry, total trade value and average close for a given symbol (e.g., CALL sp_symbol_report('BANKNIFTY')).

Trigger

trg_data4_price_audit: AFTER UPDATE on data4; when close_price changes, inserts an audit row into data4_audit with old/new values and timestamp.

These constructs improve reusability, encapsulate business rules, and provide automated auditability for price updates.

```
data  stored  trigger  correlated  indexing  join  views  schema
USE fno_tae2_db;

-- View 1: reusable reporting layer for daily volatility analysis.
CREATE OR REPLACE VIEW vw_volatility_report AS
SELECT d1.trade_date, d1.symbol,
       d1.underlying_close, d1.futures_close,
       d2.current_underlying_daily_volatility,
       d2.current_futures_daily_volatility,
       d2.applicable_daily_volatility,
```

```
USE fno_tae2_db;

-- Store the old and new close price whenever a data4 row changes.
CREATE TABLE IF NOT EXISTS data4_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    data4_id INT NOT NULL,
    old_close_price DECIMAL(18, 4) NOT NULL,
    new_close_price DECIMAL(18, 4) NOT NULL,
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
)
```

audit_id	data4_id	old_close_price	new_close_price	changed_at
1	1	57176.4000	57176.4100	2026-09-24 01:53:36

Indexing & Query Optimization — Key Outcomes

Targeted indexes were created on commonly filtered and joined columns to reduce scan costs and accelerate analytics.

Index	Purpose
idx_data1_symbol_date (data1.symbol, trade_date)	Fast symbol-date lookups
idx_data2_annualised_volatility (data2.annualised_volatility)	Volatility filters
idx_data3_symbol_expiry (data3.symbol, expiry_date)	Expiry & contract searches
idx_data3_trade_value (data3.trade_value)	High trade-value queries

- **Optimization Workflow**
 - Add indexes on JOIN / WHERE columns
 - Run EXPLAIN to inspect access methods and estimated rows
 - Iteratively refine queries and indexes to reduce execution cost
- **Key Point**

Proper indexing dramatically reduces scanned rows and improves response time for analytical queries.

Conclusion

What the project demonstrates?

Raw NSE F&O datasets are cleaned and converted into a normalized MySQL database. The implementation then demonstrates relational queries, reusable views, a stored procedure, an audit trigger, indexing and verification.

Database

MySQL 8.x Database:
fno_tae2_db

Core tables

- data1
- data2
- data3
- data4

Thank You!!

Main SQL features

- JOIN • correlated queries • views
• procedure • trigger
• indexes